

WebGIS 瓦片地图引擎实现之——地图瓦片加载计算原理介绍

1. 背景

1.1 地图瓦片之前

在地图瓦片技术使用之前，用户使用在线地图，一般都是客户端把将要显示的地理范围传送到服务端，服务器端将地理范围内的地理数据都查询出来，然后在服务端按照预先定义的专题地图的配置样式渲染成地图图片，再返回给客户端浏览，这也就是 OGC

<https://www.ogc.org/standards/wms/introduction>（地理信息联盟）标准的 WMS

<https://www.ogc.org/standards/wms>（Web Map Service）服务。

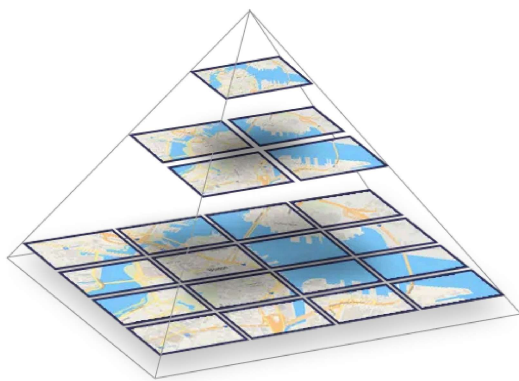
在 Web 时代 WMS 服务有几个大的缺点，由于用户浏览的地理范围是不确定性质的，获取范围内的数据是不确定，有时候数据会很大，数据从服务端的数据库到客户端的过程中，服务器 IO 操作和网络传输就很耗时；服务端获取数据后会进行数据渲染出图，占用大量 CPU 资源；用户频繁操作地图，服务端出图传输给客户端浏览过程中耗时非常长。

1.2 什么是地图瓦片

为了解决这些问题，谷歌地图率先提出了 TMS

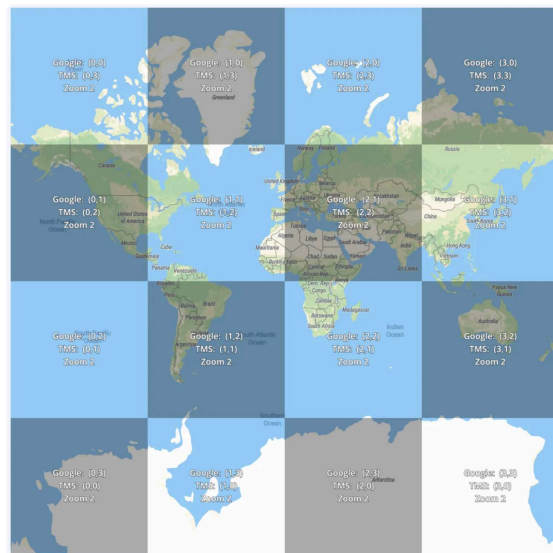
https://wiki.osgeo.org/wiki/Tile_Map_Service_Specification（Tile Map Service）服务，预先在服务端分层（金字塔模型）切片全量渲染，提出了 Web-Mercator 投影

https://en.wikipedia.org/wiki/Web_Mercator_projection，按照地图不同的显示级别切成了瓦片坐标的图片。



<<https://www.maptiler.com/img/tools/pyramid2.png>>

金字塔模型



<<https://www.maptiler.com/google-maps-coordinates-tile-bounds-projection/#3/44.64/54>>

瓦片坐标

用户每次访问时候，根据当前的地理范围映射到瓦片坐标的图片索引（XYZ），然后从后端请求这些图片索引，客户端拿到图片后按照顺序依次渲染图片即可，整个过程中用户体验有明显的提升。很快 TMS 服务成为了 WebGIS 工业标准，在该技术的推动下，OGC

<<https://www.ogc.org/standards/wms/introduction>>（地理信息联盟）也发布了基于 TMS 的 WMTS <https://en.wikipedia.org/wiki/Web_Map_Tile_Service> 服务规范，国内外很多地图厂商也基于该技术生产了自己的切片地图服务。

1.3 栅格瓦片与矢量瓦片

栅格瓦片就是上述所说的地图瓦片，只是瓦片的数据形式是图片，以矢量数据组织的瓦片是矢量瓦片。栅格瓦片技术使用很广泛，也存在一些问。瓦片预切图时间长，数据更新不方便，按金字塔模型进行切图很耗时，每进入下一级别图片成 2^{zoom} 指数级别增加；服务端资源要求高，切片过程需要极大服务端渲染和切图的计算资源，存储数据冗余，在存储了原始数据外，还存储了冗余的切片数据。栅格瓦片地图样式单一，因为最终渲染是图片数据，不能满足自定义地图风格。

因上面这些问题 Mapbox <<https://www.mapbox.com/>> 提出了 MVT 矢量瓦片切片技术，地图定制化极为方便，该技术目前最受欢迎，除二维地理数据切片外三维数据也深受此技术的影响诞生了 3DTiles <<https://github.com/CesiumGS/3d-tiles/tree/main/specification>>。当然说到这里这里瓦片预切图时间长与数据存储冗余并没有完全得倒解决，这里不再延续展开目前有的方案（数据动态切片与不切片）。

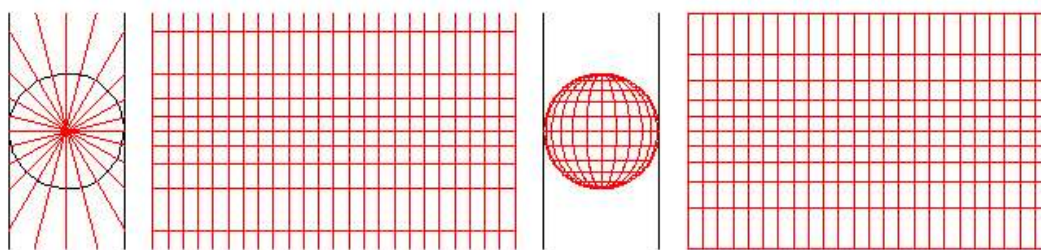
2. 瓦片地图生产

为了更好的使用瓦片服务，我们需要了解瓦片地图的生产过程，这里我们以栅格瓦片为例。下面分别以重要节点来简述这几个步骤。

2.1 数据投影及配图

2.1.1 数据投影

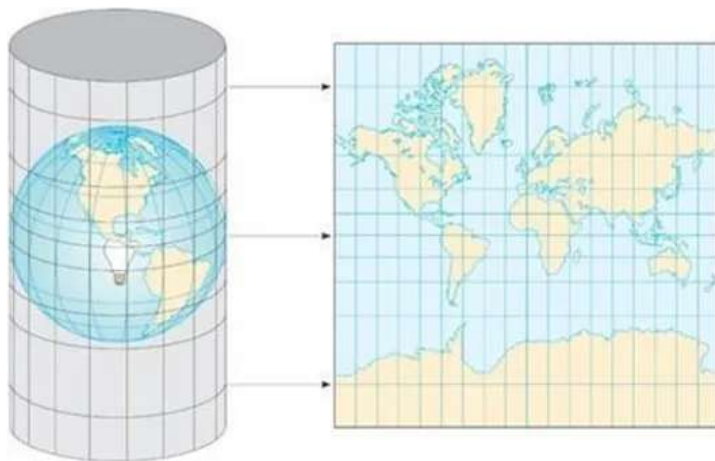
数据库中地理数据存储的形式一般都是大地坐标系（以纬度、经度和高度表示坐标元素），一般使用经纬度描述地球上点的位置，但是对于地图地理数据在二维平面内展示的场景，需要通过投影的方式将三维空间中的点映射到二维空间中，地图投影需要建立地球表面点与投影平面点的——映射关系，映射方法有很多种，各有各的优缺点，互连网地图中常用 [Web 墨卡托投影](https://en.wikipedia.org/wiki/Web_Mercator_projection) [<https://en.wikipedia.org/wiki/Web_Mercator_projection>](https://en.wikipedia.org/wiki/Web_Mercator_projection)，它是墨卡托投影 [<https://en.wikipedia.org/wiki/Mercator_projection>](https://en.wikipedia.org/wiki/Mercator_projection) 的变体。



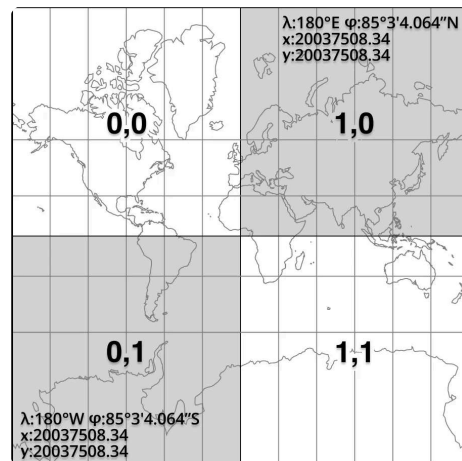
墨卡托投影过程示意图

Web 墨卡托投影是一个球面墨卡托的投影，它具有与球面墨卡托相同的属性，子午线都是等距的垂直线，角度在局部是正确的，而面积会随着离赤道越来越远而膨胀，以至于极地区域被严重夸大。由于 Web 墨卡托投影指定了在 [WGS 84](https://en.wikipedia.org/wiki/World_Geodetic_System) [<https://en.wikipedia.org/wiki/World_Geodetic_System>](https://en.wikipedia.org/wiki/World_Geodetic_System) 椭球面模型上给出的测量坐标，但在投影的时候定义在球面上的，因此会有偏离，但在小比例尺民用场景精度可忽略不计，它主要的优点还是计算简单，数学运算复杂度低。

在缩放层级为 0 时，投影的过程就是将球面投影到一个正方形的平面上，这个平面就是世界坐标调整为左上角为 (0, 0)，右下角为 (256, 256) 的正方形，假设单位为像数，那地图投影在一个 256px * 256px 的图片上。



Web 墨卡托投影



球面墨卡托投影

因为将极点投影在无穷远处，所以使用 Web 墨卡托投影的地图无法显示极点，数据覆盖范围在经度 $(-180^{\circ} \sim 180^{\circ})$ ，纬度 $(-85.051129^{\circ} \sim 85.051129^{\circ})$ 之间，投影的地图也是正方形。

2.1.2 地图配图

对数据进行投影之后，就需要根据不同的比例尺级别对数据进行配置，不同比例尺级别，显示的内容详略是不一样的，对数据进行分层级进行配图，比如层级大于 18 的时候，才显示一些街道级别的道路和 POI 等。再根据数据的不同类型进行样式配置，比如海洋填充显示为淡蓝色，铁路显示为黑白相间的线段等。

数据的分层配图比较简单，但要配出一张好看的图需要处理好很多细节，专业的桌面软件有 ArcGIS、QGIS 等，感兴趣可体验简化在线版 [Maputnik](https://maputnik.github.io/editor/#2.83/19.57/32.47)

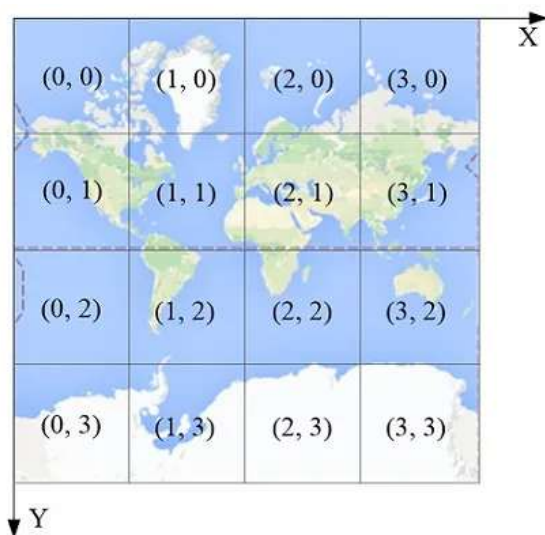
[<https://maputnik.github.io/editor/#2.83/19.57/32.47>](https://maputnik.github.io/editor/#2.83/19.57/32.47) 配置矢量瓦片样式，这里不再展开。

2.2 地图切片

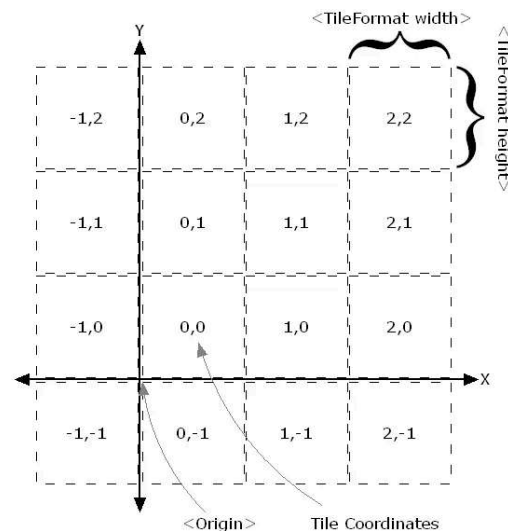
这里以谷歌地图规范的 TMS 为例，各地图厂商切片方式略有不一样，但都是基于 TMS 大同小异。

2.2.1 瓦片坐标系统

与谷歌地图相同的有高德地图、OpenStreetMap，一般都采用这种瓦片坐标系，将 Web 墨卡托投影地图的左上角作为瓦片坐标系起点，往左方向为 X 轴，X 轴与北纬 85.05° 重合且方向向左；往下方向为 Y 轴，Y 轴与东经 180° 重合且方向向下。瓦片坐标最小等级为 0 级，此时平面地图是一个像素为 256×256 的瓦片。在某一瓦片层级下，瓦片坐标的 X 轴和 Y 轴各有 2^{zoom} 个瓦片。



TMS 瓦片坐标系

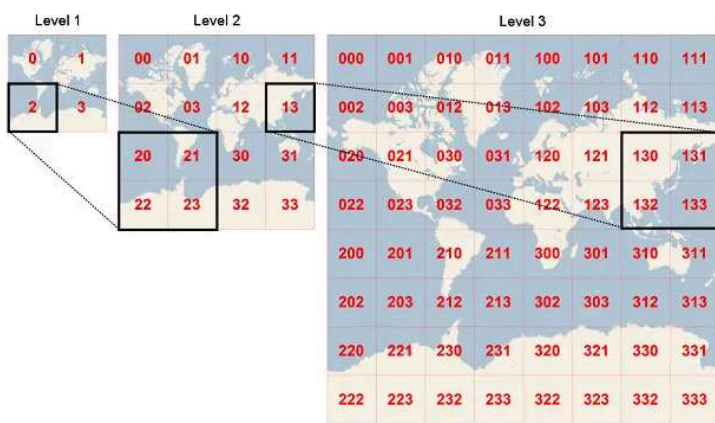


https://wiki.osgeo.org/wiki/Tile_Map_Service_Specification

WMTS 瓦片坐标系

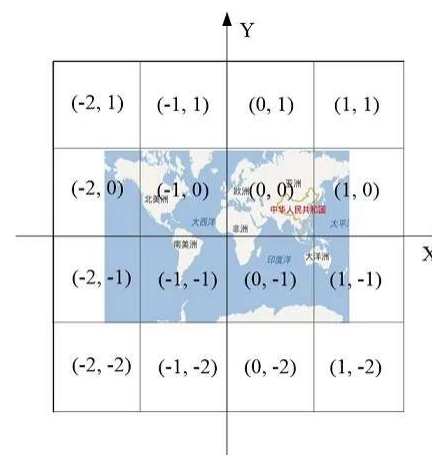
WMTS 规范中，Web 墨卡托投影地图的左下角作为瓦片坐标系起点，也就是坐标系原点为东经 180°，南纬 85.05°，X 轴向右，Y 轴向上，瓦片坐标最小等级也从 0 级开始，常见使用该规范的地图服务有天地图、腾讯地图、ArcGIS、超图等。

需要注意的是，微软的必应地图使用的编码规范，Z 的定义与谷歌相同，同一层级的瓦片不用 XY 两个维度表示，而只用一个整数表示，该整数服从四叉树编码规则；百度地图的，Z 从 1 开始，在最高级就把地图分为四块瓦片，XY 的原点在经纬度为 0 的位置，X 从左向右，Y 从下向上。



<https://docs.microsoft.com/en-us/bingmaps/articles/bing-maps-tile-system?redirectedfrom=MSDN>

必应地图



百度地图

2.2.2 坐标转换过程

在了解了数据投影后，我们需要将经纬度坐标转换为瓦片坐标，Web 墨卡托的投影计算公式如下图所示

$$x = \left\lfloor \frac{256}{2\pi} 2^{\text{zoom level}} (\lambda + \pi) \right\rfloor \text{ pixels}$$

$$y = \left\lfloor \frac{256}{2\pi} 2^{\text{zoom level}} \left(\pi - \ln \left[\tan \left(\frac{\pi}{4} + \frac{\varphi}{2} \right) \right] \right) \right\rfloor \text{ pixels}$$

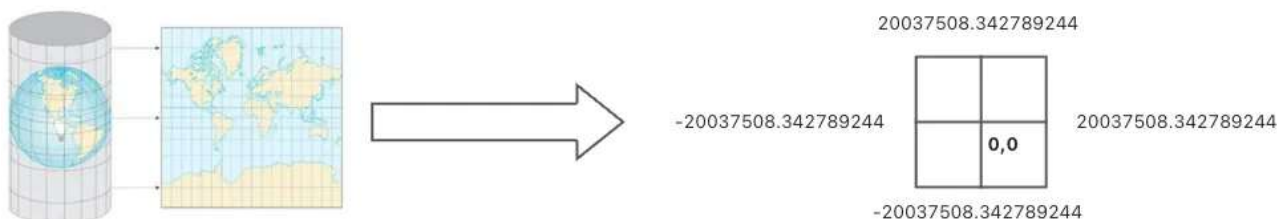
<https://zh.wikipedia.org/wiki/Web%E5%A2%A8%E5%8D%A1%E6%89%98%E6%8A%95%E5%BD%B1>

Web 墨卡托的投影计算公式

公示中 λ 是用弧度表示的经度，而 φ 是用弧度表示的纬度。经纬度坐标转换为瓦片坐标其推导过程为 经纬度 => 米 => 像素坐标 => 瓦片坐标

经纬度 => 米

将经纬度转为球面 Web 墨卡托投影的米制单位，投影到正方形



TypeScript | 运行代码 复制代码

```
1  const LatLongToMeterXY = (longitude: number, latitude: number) => {
2    // 地球的周长的一半 20037508.342789244 单位米
3    const circumferenceHalf = Math.PI * 2 * 6378137 / 2.0
4    const mx = longitude * circumferenceHalf / 180
5    const temp = Math.log(Math.tan((90 + latitude) * (Math.PI / 360.0))) / (Mat
6    const my = circumferenceHalf * temp / 180
7
8    return { mx, my }
9  }
10
11  // [-20037508.342789244, -20037508.342789244, 20037508.342789244, 20037508.34
```

米 => 像素坐标



TypeScript | 运行代码 复制代码

```

1  const MetersToPixelXY = (mx: number, my: number, zoom: number, tileSize = 256
2    // 地球的周长的一半 20037508.342789244 单位米
3    const circumferenceHalf = Math.PI * 2 * 6378137 / 2.0
4
5    // 米/每像素
6    const resolution = Math.PI * 2 * 6378137 / (tileSize * Math.pow(2, zoom))
7
8
9    const px = (mx + circumferenceHalf) / resolution
10   const py = (my + circumferenceHalf) / resolution
11
12   return { px, py }
13 }

```

像素坐标 => 瓦片坐标



TypeScript | 运行代码 复制代码

```

1  const PixelXYToTileXY = (px: number, py: number, tileSize = 256) => {
2    const tx = Math.floor(px / tileSize);
3    const ty = Math.floor(py / tileSize);
4
5    return { tx, ty }
6  }

```

2.2.3 经纬度与瓦片坐标互转

根据上面的推导过程，很容易算出经纬度坐标转瓦片坐标，瓦片坐标转经纬度坐标的公式，转换公式如下：

经纬度 => 瓦片坐标

$$x = \left\lfloor \frac{lon + 180}{360} \cdot 2^z \right\rfloor$$

$$y = \left\lfloor \left(1 - \frac{\ln \left(\tan \left(lat \cdot \frac{\pi}{180} \right) + \frac{1}{\cos \left(lat \cdot \frac{\pi}{180} \right)} \right)}{\pi} \right) \cdot 2^{z-1} \right\rfloor$$

https://wiki.openstreetmap.org/wiki/Slippy_map_tilenames

Slippy map tilenames - Latlon to tile

TypeScript | 运行代码 复制代码

```

1  const LatLongToTileXY = (longitude: number, latitude: number, zoom: number) =
2    // 地球的周长的一半 20037508.342789244 单位米
3    const circumferenceHalf = Math.PI * 6378137;
4    const mx = (longitude * circumferenceHalf) / 180;
5    const temp = Math.log(Math.tan((90 + latitude) * (Math.PI / 360.0))) / (Mat
6    const my = (circumferenceHalf * temp) / 180;
7    // 米/每像素
8    const resolution = (Math.PI * 2 * 6378137) / (tileSize * Math.pow(2, zoom))
9
10   // px = (mx + circumferenceHalf) / resolution
11   const px = ((longitude + 180) / 360) * Math.pow(2, zoom) * tileSize;
12   // py = (my + circumferenceHalf) / resolution
13   const py = ((circumferenceHalf * temp) / 180 + circumferenceHalf) / (Math.P
14
15   // tx = Math.floor(px / tileSize)
16   const tx = Math.floor(((longitude + 180) / 360) * Math.pow(2, zoom));
17   // ty = (1 - arsinh(tan(latitude * π / 180)) / π) * Math.pow(2, zoom)
18   // 双曲正弦函数 arsinh(x) => ln(x + (x^2 + 1)^0.5)
19   const ty = Math.floor(
20     ((1 -
21       Math.log(
22         Math.tan((latitude * Math.PI) / 180) +
23         1 / Math.cos((latitude * Math.PI) / 180)
24       ) /
25       Math.PI) /
26     2) *
27     Math.pow(2, zoom)
28   );
29
30   return { tx, ty };
31 };

```

瓦片坐标 => 经纬度

$$lon = \frac{x}{2^z} \cdot 360 - 180$$

$$lat = \arctan \left(\sinh \left(\pi - \frac{y}{2^z} \cdot 2\pi \right) \right) \cdot \frac{180}{\pi}$$

Slippy map tilenames - Tile to latlon

TypeScript | 运行代码 复制代码

```
1  const TileXYToLatLong = (tx: number, ty: number, zoom: number) => {
2      const longitude = (x / Math.pow(2, zoom)) * 360 - 180;
3
4      const tmp = Math.PI - (2 * Math.PI * y) / Math.pow(2, zoom);
5
6      // latitude = (180 / Math.PI) * Math.atan(sinh(tmp))
7      // 双曲函数sinh(x)=(e^x - e^(-x)) * 0.5
8      const latitude = (180 / Math.PI) * Math.atan(0.5 * (Math.exp(tmp) - Math.e
9
10     return { longitude, latitude };
11 }
```

2.3 瓦片地图服务发布

切片完成后的瓦片文件在一个大目录下，一般每个缩放层级是一个目录，每列是一个子目录，并且该列中的每个瓦片是一个文件，每个瓦片文件的路径名一般为缩放层级 + 列号 + 行号，比如 /z/x/y.png 这种形式，客户端通过拼接服务的域名加文件名来请求对应的瓦片。一般服务器考虑到并发请求原因会对地图瓦片的服务器会提供几个子域名，比如 OSM 地图服务：

- A 子域名：
<https://a.tile.openstreetmap.org/14/13659/6746.png>
<<https://a.tile.openstreetmap.org/14/13659/6746.png>>
- B 子域名：
<https://b.tile.openstreetmap.org/14/13659/6746.png>
<<https://b.tile.openstreetmap.org/14/13659/6746.png>>



通过上面的网址都可以获得这个瓦片图片，从地址中可以看出，这个瓦片的列 X 是 6745，行 Y 是 3103，缩放曾经 Z 是 13。最终我们就可以把这些瓦片文件通过静态文件服务器或 CDN 发布出去给客户端使用。

3. 瓦片地图服务解析

在了解完瓦片生产过程后，我们能明白数据是怎么投影的及地图是怎么切片的，对客户端加载解析瓦片服务的理解更加容易了。客户端要想渲染出地图有这么几个简单流程：

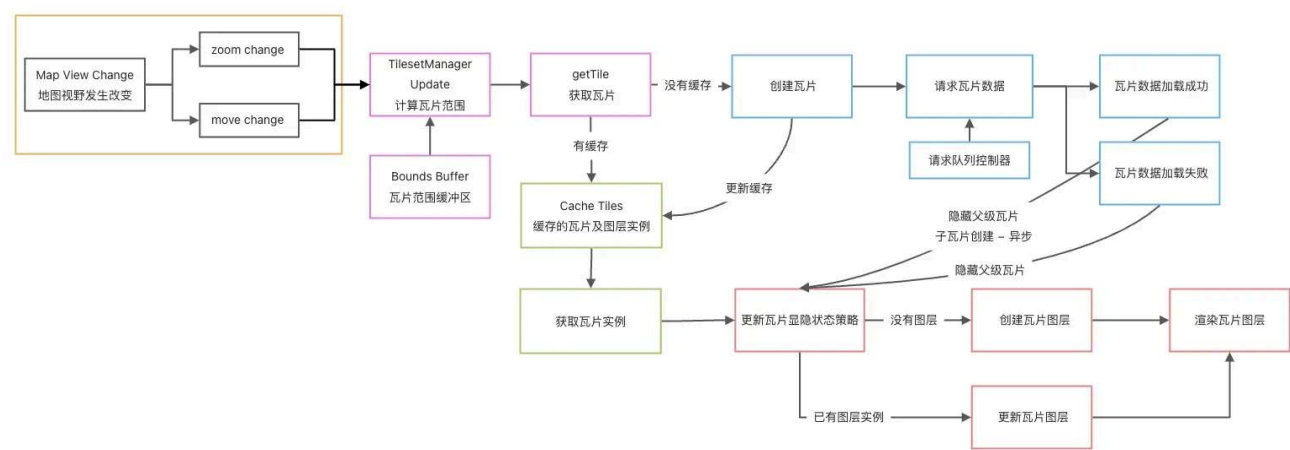
- 获取当前地图的地理范围及当前地图的缩放层级
- 通过上面两个参数计算出所有瓦片坐标（瓦片索引）
- 通过瓦片的服务地址和瓦片坐标，拿到所有瓦片的数据（栅格数据或矢量数据，如果是栅格瓦片服务也就是图片）

- 按顺序拼接好瓦片渲染出来

3.1 瓦片服务解析流程设计

为了保证瓦片数据解析流程高可用我们需要满足以下几重要点：

- 栅格瓦片和矢量瓦片服务都能使用
- 优化性能，避免频繁请求瓦片数据，瓦片数据能够缓存
- 体验优化，瓦片更新时支持渐近更新，避免白屏网格
- 支持设置缓冲区，提前预先加载瓦片
- 支持对称子午线地图重复，瓦片连贯显示



瓦片服务的解析、加载、使用流程设计

更多的设计实现详见 [L7 栅格与矢量瓦片服务解析设计](#)

<https://www.yuque.com/antv/l7/up4re5>，这里不再展开赘述。

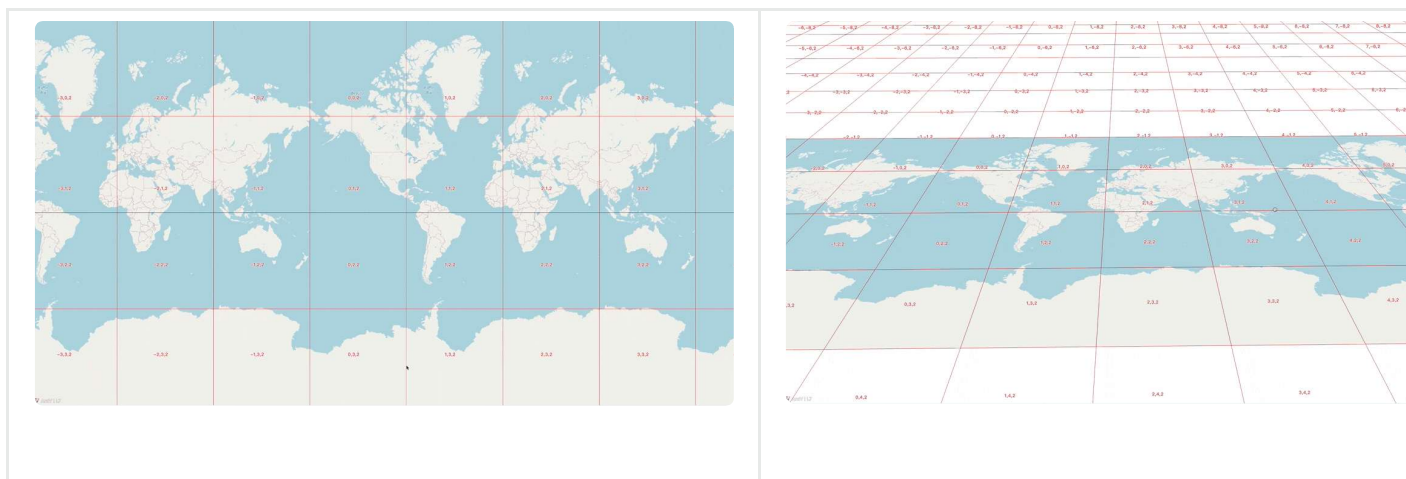
3.2 瓦片数据解析实现特性

3.2.1 支持瓦片增量更新，避免加载闪烁



3.2.2 支持对称子午线地图重复，瓦片连贯显示





4. 写在后面

我们已经完成了瓦片服务的解析、加载、使用，接下来就是渲染流程了，渲染流程主要是将拼接好的瓦片数据渲染出来，除此之外也有其它的细节考虑点（比如矢量瓦片数据的特殊处理及数据拾取等），因篇幅有限这里不再展开，见后续专题内容。